

Audion Media Tools - пользовательская карта

Audion Media Tools - портативная Windows-оболочка для реальной работы с контентом: скачать, проверить, перекодировать, подготовить к монтажу, упаковать для заливки, сохранить в архив или аккуратно разобрать аудио.

Проект сохраняет дух старой CLI-версии с продуманными пресетами, но переносит управление в GUI: выпадающие списки, пины, чекбоксы, радиокнопки, кэш путей, терминал операции и явные команды.

FFmpeg и драйвер NVIDIA

Новее не значит лучше. Каждая сборка FFmpeg компилируется под конкретную версию заголовков NVENC, и каждая из них требует своего минимума драйвера. Поставьте самую свежую на драйвер постарше — аппаратное кодирование не ускорится, а перестанет работать.

Сборка FFmpeg	Заголовки NVENC	Минимальный драйвер NVIDIA (Windows)
9.0.1	ffnvcodec n13.1.15.0	610.0
8.0.1	ffnvcodec n13.0.19.0	570.0
7.1.1	ffnvcodec n13.0.19.0	570.0
7.1	ffnvcodec n12.2.72.0	551.76

Обратите внимание на третью строку: 7.1.1 собрана теми же заголовками, что и 8.0.1, и требует те же 570.0 — «откатиться на версию назад» на старом драйвере не даёт ничего. Помогает переход на 7.1 без патча.

Поэтому установщик подбирает сборку по вашему драйверу, а не берёт последнюю. Версии выше прочитаны из README самой сборки, пороги драйверов — из README nv-codec-headers.

Если видеокарты NVIDIA нет, всё это неважно: ставится последняя сборка, а кодирование идёт на процессоре.

Какая сборка идёт в поставке: 8.0.1. Это осознанный выбор, а не забытое обновление.

Большинство машин для монтажа и кодирования сегодня живут на драйверах примерно с 571 по 609; ветка 610 стоит у считанных единиц. И 8.1.x, и 9.x требуют именно её — поставить их значит заявить аппаратное ускорение NVIDIA и не дать его большинству тех, кому оно обещано. В 8.0.1 есть всё, что используют эти программы, и она работает на тех драйверах, которые у людей действительно стоят.

Быстрый запуск

```
launcher_gui.cmd
```

Если окружение не собрано:

```
builder_main.cmd
```

В builder есть отдельные пункты установки FFmpeg из BtbN и Gyan, yt-dlp и 7-Zip в портативную папку **Tools**.

GUI запускается локально, обычно на <http://127.0.0.1:8080/>.

Канонические названия Workbench

Workbench использует единый публичный словарь Audion Image Tools во всех проектах. Кнопки всегда расположены и называются одинаково: **Источник**, **Добавить файл...**, **Назначение**, **Сбросить**, **Удалить**, **Список**.

Сбросить возвращает проектные **Source** / **Transcoded** и не удаляет файлы; **Удалить** очищает текущие **Источник** и **Назначение** только после подтверждения. В английском интерфейсе точные названия: **Source**, **Add file...**, **Target**, **Reset**, **Delete**, **List**. Варианты **Цель**, **Очистить**, **Destination** и **Clear** для этих элементов Workbench не используются.

Рабочая логика

Верхний канонический Workbench задаёт глобальные пути:

- **Источник** - выбрать папку Source.
- **Добавить файл...** - выбрать один медиафайл как Source.
- **Назначение** - выбрать OUT.
- **Сбросить** - вернуть проектные **Source** и **Transcoded**, не удаляя файлы.
- **Удалить** - после подтверждения очистить текущие Source и OUT.
- **Список** - вывести в терминал имена файлов текущего Source.
- **Probe Source** - один раз запускает ffprobe и выводит краткую строку параметров.
- **Очистить Source** - очищает выбранную папку источника после подтверждения.
- **Очистить OUT** - очищает выбранную папку назначения после подтверждения.

Пути выбираются кнопками рядом с полями. Сами dropdown-поля read-only: они показывают кэш последних путей и избранное. Пин закрепляет путь сверху. По умолчанию используются **Source** и **Transcoded**.

Как запускать операцию

1. Выбери страницу операции слева.
2. В верхней строке страницы увидишь **Назад**, название операции и кнопку запуска справа.
3. Выбери профиль.
4. При необходимости поменяй отдельные параметры.
5. Нажми кнопку запуска в верхней строке.

Профиль задаёт готовую базовую конфигурацию, а поля ниже позволяют точно её изменить. Чекбоксы по умолчанию пустые, чтобы GUI не делал скрытых предположений.

Кнопка **Команда** в шапке терминала строит предпросмотр текущей GUI-команды через тот же backend, но в dry-run режиме. Это удобно для проверки длинной FFmpeg/yt-dlp команды перед реальным запуском.

Страницы

Диагностика

- **Инвентаризация** считает рабочие папки.
- **Самопроверка** проверяет FFmpeg, ffprobe, yt-dlp и Deno.
- **Profile Doctor** сверяет GUI-операции, старые **Scripts*.cmd** wrapper-ы и общий каталог **config\script_profiles.yaml**.
- **Preset Test Matrix** прогоняет пресеты на коротком 1-секундном fixture-файле: CPU, QSV, audio, remux, FPS, LUT, storage и delivery. Недоступная NVIDIA/CUDA машина должна давать **MISS**, а не ломать весь проект.
- **Hardware Capabilities** запускает короткие smoke-тесты CPU, QuickSync/QSV, CUDA/NVENC и AMD/AMF для H.264, HEVC и AV1, плюс SVT-AV1 и dav1d. Если железа или драйвера нет на этой машине, это отмечается как **MISS**, а не как поломка проекта.

Эту страницу стоит открывать первой, если кажется, что пакет "не видит" инструменты.

Для полного реального приёмочного прогона используются

system_core\smoke_media_operations.py (GUI/backend-контракт) и

system_core\smoke_cli_profiles.py (CLI wrappers и **ffprobe**). Сетевые YouTube-профили во втором тесте строят команду без скачивания; **yt-selftest** выполняется реально.

Скачивание / YouTube

Работает через yt-dlp без старого отдельного MediaDownloader. Логика скачивания перенесена в проектный backend.

Основные элементы:

- **URL** - одна ссылка.
- **Файл ссылок** - текстовый файл, обычно **Download\urls.txt**.
- **Источник** - одна ссылка или batch-файл.
- **Форматный профиль** - best MP4, safe MP4, upload ready, custom video/audio.
- **Разрешение**, **Видео кодек**, **Формат аудио**, **Контейнер**.
- **Тюнинг скачивания** - повторы, IPv4, метаданные и обложка, без плейлиста.
- **Субтитры** - ручные, авто, встроить.
- **Потоки** - параллельные фрагменты.

Архив скачиваний по умолчанию выключен.

WebM сейчас оставлен как входной, скачиваемый и remux/download-сценарий, но не как отдельная массовая цель кодирования. Если появится конкретная доставка под WebM/VP9/AV1, её лучше добавлять отдельным профилем с понятными параметрами качества, контейнера и аудио.

Аудиопотоки

Страница для извлечения, ресемплинга, нормализации и упаковки аудио.

Доступные группы:

- **Режим** - **Извлечь аудио**, **Звук в видео** или **Конвертировать аудио**.
- **Режим работы** - компактные кнопки: Copy/remux, lossless/PCM, lossy delivery и анализ; кнопка сразу меняет параметры ниже.
- **Форматы входа** - WAV, W64, FLAC, M4A/M4B, AAC, MP3, Opus/OGA/OGG, AIF/AIFF, CAF, AU/SND, AC3, E-AC3, DTS, MKA, APE, WV, TAK, TTA, MPC, AMR/AWB, SPX, DSF/DFF и видео-контейнеры MP4/M4V/MOV/MKV/MXF/AVI/WEBM/MPG/MPEG/MTS/M2TS/TS/3GP.
- **Потоки** - первый аудиопоток, все аудиопотоки, поток по индексу или язык по тегу.
- **Формат** - Copy, WAV, FLAC, ALAC, M4A/AAC, MP3, Opus.
- **Контейнер Copy** - Auto, M4A, MKA или родной файл кодека для извлечения без перекодирования.
- **Контейнер видео** - для режима **Звук в видео / video copy**: исходный контейнер, MOV, MKV, MP4 или MXF.
- **Ресемплер** - SoX/libsoxr 100% quality (**precision=33**) для всех изменений частоты.
- **Частота дискретизации** - Copy/source, 44.1/88.2/176.4 kHz и 48/96/192 kHz.
- **Разрядность** - Copy/source, 16-bit, 24-bit, 32-bit float.
- **Битрейт аудио** - 192/256/320/384 kbps, только для AAC/MP3; 384 kbps доступен только AAC, потолок MP3 - 320 kbps.
- **MP3 / LAME** - Best VBR V0 / Extreme (~245 kbps), Insane CBR 320 или ручной bitrate. Та же шкала работает на страницах кодирования при **Аудио = MP3** и в CLI через **AUDION_MP3_LAME_PRESET**.
- **Мастеринг** - LUFDS выключен, только отчёт, one-pass loudnorm или two-pass loudnorm; цели -14, -16, -23.
- **Каналы** - сохранить, stereo, mono, dual mono и **5.1 Matrix**.

Базовый аудиобитрейт - 384 kbps. Для M4A есть отдельный профиль 256 kbps. После аудио-операций дополнительно пишется **audio_report.md/json** с таблицей: исходный поток,

параметры входа, режим обработки, LUFFS и путь результата.

Source обходится рекурсивно. OUT сохраняет структуру подпапок Source, чтобы

`Source\Day1\clip.mov` и `Source\Day2\clip.mov` не сваливались в одну плоскую папку и не конфликтовали по имени. Результаты пишутся в папку операции/профиля, а не через suffix в имени файла: например `Transcoded\Remux\remux_mkv_to_mp4_video_audio\Day1\clip.mp4`. Если выбран кастомный OUT, он просто заменяет `Transcoded`, а внутренняя структура остаётся такой же.

Режим `Звук в видео / video copy` создаёт новый видеофайл в `Transcoded\AudioInVideo`: видеопоток копируется без перекодирования (`-c:v copy`), а выбранный аудиопоток меняется внутри контейнера. Это основной режим для задач "поменять битность/частоту дискретизации аудио в видео, не отстёгивая дорожку и не трогая изображение". Для PCM/FLAC безопаснее выбирать MOV или MKV; MP4 подходит для AAC/MP3/copy.

Архив

Для долговременных мастер-копий:

- FFV1
- x264 lossless

Это отдельная архивная категория, не смешанная с обычными lossy/storage задачами.

Монтажные кодеки

Промежуточные форматы для NLE:

- ProRes Proxy / LT / 422 / HQ в MOV или MXF
- DNxHR HQ / HQX в MOV или MXF

Пресеты сгруппированы как матрица `ProRes · MOV`, `ProRes · MXF`, `DNxHR · MOV`, `DNxHR · MXF`. Кодек и контейнер также можно выбирать вручную. Для MXF доступны PCM 16/24-bit или вывод без аудио; несовместимые COPY/AAC/MP3/FLAC/PCM float блокируются в GUI и проверяются backend. FLAC недоступен и для MOV: этот контейнер его не держит, для FLAC нужен MP4 или MKV.

Pixel format на этой странице не выбирается: его задаёт сама монтажная цель. ProRes всегда пишется как 10-bit 4:2:2 (`yuv422p10le`), DNxHR HQ - как 8-bit `yuv422p`, DNxHR HQX - как 10-bit `yuv422p10le`. Чтобы поменять разрядность, выбирают другую цель, а не отдельное поле.

Хранение

Качественные компактные файлы для библиотеки или передачи:

- x264 / H.264
- x265 / HEVC
- SVT-AV1

Базовые значения: CRF/CQ 14, audio 384 kbps. Можно выбрать контейнер, разрешение, pix_fmt, preset кодера и тюнинг кодирования. Тюнинг относится к x264/x265 и показывается только для

Кодер = CPU.

Доставка / публикация

Профили для просмотра, отправки и заливки:

- Review H.264
- Upload H.264
- Upload HEVC 10-bit
- SVT-AV1 delivery

Аппаратный способ кодирования выбирается не профилем, а отдельным блоком **Кодер**: CPU, NVENC, AMF, QSV. Стек декодирования вынесен отдельно: Auto, CUDA, QuickSync, AMD/D3D11VA, dav1d AV1.

Выбранный **pix_fmt** действует и на аппаратных кодерах: он переводится в родной формат (**yuv420p10le** → **p010le**, для QSV **yuv420p** → **nv12**). Сочетания, которые кодер не умеет — например 10-bit на H.264 NVENC/QSV/AMF, — preflight помечает как ERROR до запуска FFmpeg, вместо того чтобы молча записать другой формат.

Упаковка / remix

Меняет контейнер или состав потоков без перекодирования, где это возможно. Source обходится рекурсивно, формат каждого файла определяется автоматически, а пользователь выбирает пайплайн и целевой контейнер.

- **Video** - оставить только видеотреки.
- **Audio** - отделить аудиотреки в M4A, ALAC/M4A, MKA, AAC, AC3, EAC3, DTS, FLAC, WAV, MP3, OPUS или OGG.
- **Video&Audio** - скопировать видео и аудио вместе в MP4, MOV, MKV или MXF.

Несовместимые файлы пропускаются с причиной в логе: проверяется и пара контейнеров, и реальный кодек внутри файла, поэтому ProRes не уходит в MP4, а HEVC - в MXF. Same-container сору разрешён, когда меняется состав потоков: например `MP4 -> MP4 / Video` создаёт MP4 без аудио.

`faststart` означает перенос MP4 metadata в начало файла для web/playback до полной загрузки. Здесь же задаётся разворот. Камеру повернули набок ради вертикального кадра, а она записала обычный горизонтальный файл и ничего об этом не сказала — разворот записывается в контейнер, в матрицу отображения, которую читает монтажка. Ничего не декодируется и не перекодируется: файл 960×540 сохраняет свои байты, а декодер отдаёт 540×960. Хранят это MP4, MOV и MKV; MXF нет. Предлагается для камерных H.264 и HEVC.

Обрезка

Отрезает начало, конец или кусок из середины камерного файла **без перекодирования**: потоки копируются, поколение не теряется, метаданные и таймкод переезжают в результат.

Три кнопки говорят, что сделать с куском между точками. `Оставить` — оставить его, остальное убрать; какой край уйдёт, уже сказано тем, какая точка заполнена: один IN убирает начало, один OUT — конец. `Вырезать` — наоборот: кусок уходит, а начало и конец склеиваются в один файл. `Разделить надвое` — одна точка, ничего не выбрасывается.

Точки находят в трв: раздел открывает его отдельной программой. Точная перемотка, шаг ровно в кадр в обе стороны, воспроизведение назад и петля A/B, которая и есть рез — кусок играет по кругу до того, как что-то записано. `Взять A и B` переносит обе точки в поля; их можно и просто набрать. Плеер ставится один раз: `install\Install-Portable-mpv.cmd`.

Начало копируемого потока ложится на ближайший ключевой кадр перед запрошенной точкой, и журнал говорит, куда оно попало; конец считается кадрами по точной дроби частоты и попадает ровно в запрошенный кадр. Частота читается у файла, а не додумывается: `ffprobe` отдаёт `24000/1001` для 23,976 и `24/1` для честных 24,000. Телеметрия и субтитры едут вместе с резом по умолчанию, а контейнер, который их не держит, говорит об этом до запуска.

Отказы происходят до запуска, а не в середине: MP4 не примет ProRes, MXF не примет сжатый звук, а камерный RAW в MXF FFmpeg не читает вовсе. Подробности, таблицы совместимости и измерения — в `Docs\TRIM_RU.md`, `Docs\TRIM_MATRIX_RU.md` и `Docs\MEASUREMENTS_RU.md`; клавиши плеера — в `Docs\MPV_HOTKEYS_RU.md`.

FPS / частота

Исходная частота кадров не выбирается вручную: она читается из видеопотока.

GUI выбирает:

- режим **Varispeed** или **Conform**;
- целевую частоту из полного ряда: **16**, **18**, **23.976**, **24**, **25**, **29.97**, **30**, **48**, **50**, **59.94**, **60**. Дробные считаются точными дробями, а исходная частота может быть любой;
- профиль вывода: H.264 MP4, H.265 MP4, ProRes MOV/MXF, DNxHR MOV/MXF.
- для ProRes/DNxHR доступны PCM 16/24/32-float, где контейнер поддерживает, и oversampling аудио COPY / 2x / 4x через SoX/libsoxr.

Цвет / LUT

Верхнее поле выбирает LUT из кэша и папки **LUTs**. Можно добавлять файлы или папки LUT, открывать папку и пинить часто используемые варианты.

Пресеты используют стабильные имена старых CLI-скриптов, но сами wrapper-скрипты теперь идут через единый portable runner:

- LUT -> x264
- LUT -> HEVC Main10
- LUT -> ProRes
- Pre-expose + LUT
- Pregamma + LUT
- HDR to SDR Hable

В пресетах **Pre-expose + LUT** и **Pregamma + LUT** появляется поле **Гамма до LUT**. Это один и тот же технический параметр FFmpeg **eq=gamma=<число>** перед LUT, но с разными стартовыми значениями: **Pre-expose + LUT** по умолчанию даёт более светлую подготовку (**1.15**), а **Pregamma + LUT** - более тёмную (**0.85**). Параметр можно править вручную как творческий и технический контроль изображения.

Под полем показывается 18% gray preview: слева исходный серый, справа расчётный результат после pregamma до LUT. Стрелка показывает направление преобразования, а подпись сообщает, станет серый светлее, темнее или останется нейтральным. Preview использует вытянутые скруглённые прямоугольники 2.35:1, чтобы изменение считывалось как маленький кадр, а не как декоративная иконка.

Пути LUT экранируются общим helper-ом для FFmpeg filtergraph. Это важно для Windows-путей с диском, пробелами, апострофами, запятыми, точками с запятой и скобками; GUI и wrapper-скрипты используют одну и ту же логику.

Слой `Scripts\` поддерживает override-переменные `AUDION_*`, поэтому прямой CLI и GUI не расходятся по путям, битрейту, CRF/CQ, backend-выбору и dry-run. Карта старых CLI-пресетов вынесена в `config\script_profiles.yaml`.

Терминал и статус

Правая область содержит:

- короткий статус операции: ожидание, выполнение, ошибка, результат;
- общий progress bar операции;
- журнал операции с preflight, командами, stdout/stderr и `[PROGRESS]` строками FFmpeg;
- кнопку `Команда` для dry-run предпросмотра текущей GUI-команды;
- кнопки `Logs`, `Report`, `CONFIG`, `Развернуть`;
- страницу `Report` со списком последних операций, открытием OUT/report/log/JSON и повтором команды;
- многострочную командную строку под терминалом для админских утилит;
- нижнюю строку ffprobe-статуса.

Probe не запускается постоянно. Это сознательно: анализ больших папок тяжёлый, поэтому ffprobe запускается кнопкой.

Командная строка под терминалом хранит историю до 200 записей в

`config\terminal_commands.json`, поддерживает пины, выбор Shell, CWD, добавление файла/папки через picker и многострочный перенос длинных команд. Поле команды при запуске остаётся пустым: прошлые команды выбираются из истории. Кнопка `История` очищает незакреплённую историю, `Кэш` полностью сбрасывает history/pinned/last command, сохраняя Shell и CWD. Это место для ручных служебных команд; пользовательский запуск медиа-операций остаётся в левом дереве операций.

FFmpeg-прогресс в терминале относится к текущему файлу: GUI добавляет `-progress pipe:1`, когда известна длительность через ffprobe, и выводит компактные строки вида `[PROGRESS] name [####----] 42% 00:12/00:30`. Верхний progress bar показывает общий ход операции и финализируется по завершённым файлам/целям batch-a.

Параллельные кодирования

Один экземпляр GUI запускает одну медиа-операцию за раз, чтобы терминал, отчёты и progress не смешивались. Два независимых FFmpeg-процесса или две отдельные программы могут

кодировать одновременно, но они делят CPU/GPU/диск и могут заметно замедлить друг друга. Нельзя писать в один и тот же выходной файл, а для старых ручных CMD-сценариев лучше избегать общего mutable-файла вроде `LUTs\active.cube`.

Preflight и отчёты

Перед медиа-операциями GUI выполняет быстрый preflight:

- проверяет выбранный `Source` и `OUT`;
- считает подходящие входные файлы;
- проверяет выбранную LUT;
- проверяет безопасную remux-цель для каждого файла;
- сверяет наличие нужных FFmpeg encoder-ов;
- сверяет выбранный hardware backend с последним `Hardware Capabilities` cache для текущей машины;
- для YouTube проверяет `yt-dlp`, URL или файл ссылок.

Если preflight находит ошибку, команда не запускается. При этом всё равно создаётся отчёт, чтобы было видно, что именно остановило операцию.

`Hardware Capabilities` записывает `config\hardware_capabilities_cache.json`. Если cache отсутствует или был создан на другой машине, preflight не блокирует hardware backend, но предупреждает, что диагностику стоит прогнать заново. Если cache текущей машины говорит `MISS` или `FAIL`, выбранный CUDA/QSV/AMF/dav1d backend останавливается до запуска FFmpeg.

Кодер и декодер гвардятся отдельно: `Кодер` проверяется по smoke-тесту своего энкодера (`NVENC HEVC encode`, `QSV AV1 encode` и т.п.), `Декодировать через` - по своему decode-тесту (`CUDA decode`, `QuickSync decode`, `AMD/D3D11VA decode`, `dav1d AV1 decode`). Машина без NVENC может при этом свободно использовать аппаратный декод.

После каждой операции создаются:

- `batch_report.md` - читаемый отчёт;
- `batch_report.json` - машинный отчёт;
- обычный `.log` в `logs\`.

Кнопка `Report` открывает страницу последних отчётов. Из неё можно открыть каталог report, OUT, `batch_report.json`, лог операции или повторить команду с параметрами из отчёта.

User guides и разделы

Главные пользовательские guides:

- `USER_GUIDE_RU.md` - полный русский guide по GUI и операциям.
- `USER_GUIDE_EN.md` - английская версия.

Глубокие разделы в `Docs\docs\`:

- `AUDION_MEDIA_TOOLS_RU.md` - карта проекта, папки, manifest и service layer.
- `WORKSPACE_REPORTS_TERMINAL_RU.md` - Source/OUT, path cache, reports и терминал.
- `YOUTUBE_DOWNLOAD_RU.md` - yt-dlp profiles, batch, subtitles и cookies.
- `AUDIO_WORKFLOWS_RU.md` - три режима аудио и их параметры.
- `ENCODING_PIPELINES_RU.md` - архив, монтажные кодеки, хранение и публикация.
- `REMUX_FPS_LUT_RU.md` - remux, FPS, LUT и HDR-to-SDR.
- `HARDWARE_DIAGNOSTICS_RU.md` - selftest, hardware cache, Profile Doctor и Test Matrix.
- `MEDIA_KNOWN_PITFALLS_RU.md` - типовые ошибки.
- `MEDIA_SMOKE_TEST_CHECKLIST_RU.md` - проверки после изменений.

Где менять пресеты

Главная точка настройки:

```
config\tool_manifest.yaml
```

В нём описаны:

- дерево операций;
- поля GUI;
- профили;
- значения по умолчанию;
- дочерние команды запуска;
- maintenance-операции.

Код выполнения находится в:

```
system_core\services\media_service.py
```

Интерфейс:

```
system_core\ui_nicegui\app.py
```

CLI и установщики

Активный слой установки теперь сосредоточен в `builder_main.cmd` и `install\`.

- `install\download\` - кэш установочных архивов.
- `system_core\fzf.exe` - FZF для builder-меню, если установлен.
- `system_core\powershell\` - portable PowerShell для builder/helper-скриптов.
- `Tools\ffmpeg\bin\`, `Tools\yt-dlp\bin\`, `Tools\deno\`, `Tools\7zip\bin\` - внешние media-бинарники и runtime-компоненты.

Установщики media-tools работают в режиме полной замены:

- `install\Install-Portable-FFmpeg-BtbN.cmd` ставит наиболее новую BtbN win64 `gpl` сборку из release-ветки FFmpeg через `install\Ensure-7zip.ps1`; master/nightly fallback запрещён. Установщик очищает `Tools\ffmpeg\`, создаёт `Tools\ffmpeg\bin\` и кладёт весь свежий `bin` payload.
- `install\Install-Portable-FFmpeg-Gyan.cmd` требует 7-Zip и применяет driver-aware Stable policy: при NVIDIA 570+ выбирает latest Gyan Stable 8.x, при 471.41-569.xx — последний опубликованный Gyan Stable 7.1.1. GitHub используется первым, Gyan direct — резервом; cache привязан к версии и SHA-256. После проверки установщик очищает `Tools\ffmpeg\`, создаёт `Tools\ffmpeg\bin\` и кладёт `bin` payload.
- `install\Install-Portable-yt-dlp.cmd` скачивает свежий `yt-dlp.exe`, очищает `Tools\yt-dlp\`, создаёт `Tools\yt-dlp\bin\` и кладёт новый executable. После этого он ставит Deno в `Tools\deno\`, чтобы yt-dlp мог использовать внешний JavaScript runtime. Старый legacy-файл `Tools\yt-dlp\yt-dlp.exe`, если он когда-либо был, не сохраняется.
- `install\Install-Portable-7Zip.cmd` скачивает свежий 7-Zip extra pack, очищает `Tools\7zip\`, создаёт `Tools\7zip\bin\` и кладёт новый payload.
- `install\Install-Portable-PowerShell.cmd` аналогично заменяет `system_core\powershell\`.
- `install\launcher-tools-update_fzf.cmd` заменяет `system_core\fzf.exe`.

Это сделано намеренно: FFmpeg, yt-dlp, Deno, 7-Zip, portable PowerShell, FZF, `runtime\`, `wheelhouse\` и `install\download\` считаются воспроизводимыми payload-ами. Если они

устарели или повреждены, их нужно обновлять заменой, а не копированием поверх старого содержимого.

`cleanup_project.cmd` возвращает проект в lean/rebuild-состояние. Он чистит `runtime\`, `wheelhouse\`, `install\download\`, `Tools\ffmpeg\`, `Tools\yt-dlp\`, `Tools\deno\`, `Tools\7zip\`, `system_core\7zip\`, `system_core\powershell\`, `system_core\fzf.exe`, временные installer-папки, `logs\`, `report\`, `workspace\`, `data\`, `release\` и `Transcoded\`. Пользовательские `Source\`, `Download\` и `LUTs\` он не очищает.

Лицензии и релизные notices

Собственный код, скрипты и документация Audion Media Tools выпускаются под `GPL-3.0-or-later`. Полный текст лежит в `LICENSE` и `Docs\LICENSE (GPL-3.0-or-later).md`.

FFmpeg, yt-dlp, Deno, 7-Zip, PowerShell, fzf, Python Embedded и Python-пакеты сохраняют свои лицензии. Перед публикацией бинарного архива нужно собрать финальное содержимое, затем запустить:

```
system_core\license\Run-Collect-And-Deduplicate-ThirdPartyLicenses.cmd /NOPAUSE
```

Итоговый файл `licenses\THIRD_PARTY_NOTICES.md` должен соответствовать реальному staging-архиву. Для публичного бинарного релиза строки с `UNKNOWN` считаются блокером. Если в архив входит FFmpeg, на странице релиза или рядом с архивом должна быть ссылка/копия соответствующего FFmpeg source/build info; nonfree-сборки FFmpeg не используем.

Проверка после изменений

```
runtime\python.exe -m py_compile system_core\ui_nicegui\app.py
system_core\services\media_service.py
runtime\python.exe system_core\ui_nicegui\app.py --smoke
```